

Sprawozdanie z zajęć projektowych: Sieci Komputerowe

Prowadzący: Dr inż. Łukasz Maliński

Skład sekcji: Kacper Chmura, Paweł Nagórzański

Grupa: IP21

Rok akademicki: 2026 semestr 4

1 Skład sekcji wraz z podziałem obowiązków:

Kacper Chmura - weryfikacja i walidacja otrzymanego kodu, zaprojektowanie struktury ramki protokołu sieciowego, implementacja mechanizmu wykrywania opóźnień i jego sygnalizacji w GUI, przygotowanie okna konfiguracji połączenia.

Paweł Nagórzański - implementacja komunikacji TCP (klient-serwer), oprogramowanie serializacji i deserializacji danych przy użyciu QDataStream, synchronizacja parametrów UAR po nawiązaniu połączenia.

2 Opis protokołu komunikacyjnego:

Ramka protokołu sieciowego:

Typ ramki	Start	Typ ramki	Długość	Dane	CRC
Długość	1B	1B	2B	zależy	2B
PID	0xAA	0x01	56B	timestamp-Kp-Ti-Td-Tp-Umin-Umax-TrybCalk	Suma Bajtów
ARX		0x02	Zależy od długości A[] i B[]	timestamp-A[]-B[]-k-szum-ograniczenie-Umin-Umax-Ymin-Ymax	
Generator		0x03	36B	Typ-amplituda-okres-wypełnienie-StałaSkładowa	
InfoPolaczenia		0x04	22B-38B	rola, lokalneIP	
SymulacjaRegulator		0x06	24B	czas, w, u	
SymulacjaObiekt		0x07	16B	czas, y	
Reset		0x08	0B	---	

Struktura ramki protokołu sieciowego składa się z pięciu głównych sekcji:

1. Start - 1 bajt, stała wartość 0xAA oznaczająca początek komunikatu.
2. Typ ramki - 1 bajt, identyfikator przesyłanego ładunku.
3. Długość - 2 bajty, określa rozmiar pola danych.
4. Dane - pole o zmiennej długości, przechowujące właściwe parametry.
5. CRC - 2 bajty, suma kontrolna weryfikująca integralność wszystkich bajtów.

Obsługiwane typy ramek oraz ich zawartość:

- 0x01 PID - 56 bajtów: timestamp, Kp, Ti, Td, Tp, Umin, Umax, TrybCalk.
- 0x02 ARX - długość zmienna: timestamp, tablice A i B, k, szum, ograniczenie, Umin, Umax, Ymin, Ymax.
- 0x03 Generator - 36 bajtów: Typ, amplituda, okres, wypełnienie, StałaSkładowa.
- 0x04 InfoPolaczenia - od 22 do 38 bajtów: rola, lokalneIP.
- 0x06 SymulacjaRegulator - 24 bajty: czas, w, u.
- 0x07 SymulacjaObiekt - 16 bajtów: czas, y.
- 0x08 Reset - 0 bajtów, ramka pusta.

Serializacja:

Wymiana danych wykorzystuje serializację binarną za pośrednictwem klasy QDataStream. Zmienne konfiguracyjne obiektów UAR oraz próbki symulacji w każdym takcie są zapisywane do tablicy QByteArray w stałej kolejności. Proces deserializacji odczytuje bajty po stronie odbiorcy w identycznej kolejności, odtwarzając stan obiektów i parametry symulacji bez narzutu charakterystycznego dla formatów tekstowych.

Wykrywanie gubienia pakietów:

System wykrywania opóźnień bazuje na monitorowaniu zdarzeń sieciowych względem głównego zegara symulacji. Obliczenia pętli regulacji nie są wstrzymywane ani blokowane przez oczekiwanie na nadchodzące pakiety.

Kontrola synchronizacji wykorzystuje binarną flagę stanu:

- Poprawne odebranie i zdeserializowanie ramki z wartością regulowaną od instancji obiektu modyfikuje flagę na stan potwierdzający nadejście danych.
- Wyzwolenie każdego kolejnego taktu zegara symulacji odczytuje i przekazuje aktualną wartość flagi dalej.
- Flaga jest natychmiast resetowana do stanu początkowego, przygotowując system na weryfikację w kolejnym interwale.

Informacja o traceniu danych:

Wystąpienie opóźnienia lub zagubienie pakietu w sieci nie przerywa działania układu. W przypadku braku nowej próbki na czas, algorytm automatycznie wykorzystuje do obliczeń uchybu ostatnią poprawnie odebraną wartość regulowaną.

Sygnalizacja w interfejsie graficznym:

Wynik weryfikacji w każdym takcie symulacji jest emitowany do interfejsu graficznego. Dedykowana kontrolka wydajności w głównym oknie zmienia kolor w zależności od terminowości pakietów:

Zielony - dane z obiektu dotarły przed rozpoczęciem bieżącego taktu symulacji.

Czerwony - brak nowych danych, układ odnotował zgubienie pakietu.

3 Historia rozwoju programu:

- **Walidacja i weryfikacja otrzymanego kodu** - sprawdzono zgodność działania otrzymanej aplikacji z check-listą wymagań z instrukcji do projektu
- **Wykrycie i poprawa błędów przesunięcia generatora oraz resetu całki** - zidentyfikowano i naprawiono dwa celowe błędy pozostawione przez sekcję przekazującą kod
- **Zaprojektowanie struktury ramki protokołu sieciowego** - opracowano format ramki protokołu sieciowego
- **Oprogramowanie komunikacji sieciowej TCP** - zaimplementowano połączenie klient-serwer z wykorzystaniem klas Qt (QTcpSocket/QTcpServer), obsługujące nawiązywanie i zrywanie połączenia
- **Oprogramowanie binarnej serializacji i deserializacji danych** - zrealizowano zapis i odczyt danych konfiguracyjnych oraz próbek symulacji do/z QByteArray przy użyciu QDataStream, w stałej, ustalonej kolejności pól
- **Dodanie dedykowanego okna konfiguracji połączenia sieciowego** - utworzono osobne okno GUI do zarządzania połączeniem, obsługiwane przyciskami zabezpieczonymi przed przypadkowym wciśnięciem
- **Synchronizacja parametrów po połączeniu** - zaimplementowano automatyczne przesyłanie i odtwarzanie konfiguracji UAR (PID, ARX, generator) po nawiązaniu połączenia, tak aby obie instancje miały identyczny stan
- **Wdrożenie mechanizmu detekcji opóźnień i ich sygnalizacji** - dodano flagę stanu synchronizacji resetowaną w każdym takcie zegara oraz kontrolkę w GUI zmieniającą kolor (**zielony/czerwony**) w zależności od terminowości odbieranych pakietów

4 Opis trudności:

Jedną z głównych trudności był fakt, że pracowaliśmy na programie napisanym wcześniej przez inną grupę, do którego musieliśmy dopisać własną warstwę sieciową. Wymagało to najpierw dokładnego przeanalizowania obcego kodu i zrozumienia logiki jego działania, zanim mogliśmy rozpocząć pracę nad jego rozbudową o funkcje sieciowe.

Kolejnym wyzwaniem było samo programowanie funkcji sieciowych, z którym wcześniej nie mieliśmy większej styczności. Implementacja komunikacji TCP, serializacji danych oraz obsługi połączeń między klientem a serwerem wiązała się z koniecznością nauki nowych zagadnień.

Znaczącym utrudnieniem był również sam proces testowania nowych funkcji, z uwagi na ograniczone możliwości pracy w tym samym miejscu, testy przeprowadzaliśmy zdalnie, na własnych urządzeniach. Dodatkowo komunikacja sieciowa wprowadzała zauważalne opóźnienia w przesyłaniu danych, co skutkowało momentami mocno zniekształconymi (rozjechanymi w czasie) wykresami. Paradoksalnie okazało się to również pomocne, bo pozwoliło łatwiej zauważyć wpływ opóźnień sieciowych na synchronizację danych oraz zweryfikować poprawność działania mechanizmów odświeżania wykresów przy dużym i zmiennym opóźnieniu.

5 Podsumowanie nauki:

Kacper Chmura:

- Praca na cudzym kodzie
- Weryfikacja i walidacja kodu
- Projektowanie protokołów sieciowych
- Ocena jakości cudzego rozwiązania i formułowanie konstruktywnego feedbacku (code-review)
- Projektowanie GUI odpornego na błędy operatora
- Testowanie i debugowanie komunikacji sieciowej w różnych konfiguracjach połączenia

Paweł Nagórzański:

- Praca na cudzym kodzie
- Obsługa połączeń TCP wraz z serializacją/deserializacją danych przy użyciu strumieni binarnych Qt
- Wykrywanie i obsługa utraty oraz opóźnień pakietów w symulacji czasu rzeczywistego
- Testowanie i debugowanie komunikacji sieciowej w różnych konfiguracjach połączenia